Module 4

Giving the agent your team's tools

55 minutes · MCP-led

What this module is about

MCP — Model Context Protocol — is how you give the agent the tools your team already uses.

By the end of the hour:

- You've **mob-built a real MCP tool** and added it to a working server.
- You've written a **skill** that codifies one of your team's recurring tasks.
- You know when **parallel agents** earn their keep and when they don't.

Skills and parallel agents are *how MCP scales* in a team — they're the supporting shape around the load-bearing idea.

The hour

- 3 min — intro + worktrees primer
- 25 min — **MCP mob** (build `assign_route`) ← the spine
- 10 min — parallel agents (and when not to)
- 15 min — write a skill solo
- 2 min — wrap

15-minute break after.

*This is where you start being able to **trust the agent with delegated work** — because you've given it the tools, conventions, and skills it needs to do the job your way.*

MCP tool convention

Each tool: three exports plus one registration in `index.ts`.

Example (`list_vehicles`):

- `listVehiclesInputShape` — Zod input schema
- `listVehiclesDescription` — what the tool does and returns
- `listVehiclesHandler` — the work

Mirror this when adding new tools.

Description is pedagogy

The LLM reads it to decide *when* to call your tool.

Vague descriptions cause the agent to guess,
misuse the tool, or hallucinate the response shape.

Highest-leverage line per tool. Invest here.

Building `assign_route` — decisions

1. **Schema** — what fields?
2. **Maintenance** — what if vehicle is in maintenance?
3. **Capacity** — what if `loadKg > capacityKg`?
4. **Conflict** — what if vehicle is already on a route?
5. **Success** — if it succeeds, return `{ ok: true }` or updated state?
6. **Bidirectional** — what if `route.assignedVehicleId` isn't updated?

Worktrees + parallel agents

Multiple sessions — agents in different worktrees, independent file state.

Sub-agents — one session delegates scoped work via the Task tool.

Parallel is overhead. Default: one agent, one task, one session.

Two 5-min tasks → sequence them. Parallel earns its keep at 30+ min of independent work.

Write a skill

Pick one:

1. **Code review** in DispatchKit style
2. **PR description** in a fixed format
3. **Plan a feature** from a one-liner

Template: `modules/module-4/sample/skill-template/SKILL.md`

Lives at: `.claude/skills/<name>/SKILL.md`

Open Claude. Have it draft the frontmatter and body with you. Then invoke the skill three times — iterate, don't just admire it.

What you take home

The load-bearing idea: **MCP is how your team's tools become the agent's tools.**

- A working MCP server in this repo, plus the convention to grow it.
- A skill that codifies one recurring task — versioned, reviewed, reusable.
- A clear rule for when parallel agents are worth the overhead.

`.prompts/` is homework — the same shape, applied to text you reach for.

Coming up: Module 5

When AI follows the spec but the code doesn't.

- Spec-first
- The golden rule
- Bidirectional sync
- Review a real PR

15-minute break.